

CHAPTER 2 – INSTRUCTIONS: LANGUAGE OF THE COMPUTER PATTERSSON/HENNESSY

Luiz Paulo de Assis Barbosa

2.1 – Introdução

2

- Para controlar, por meio de comandos, o *hardware* do computador é preciso conhecer a ***linguagem*** que ele compreende.
- As palavras dessa linguagem são chamadas ***instruções***.
- E o vocabulário de comandos compreendidos por uma determinada arquitetura é o ***conjunto de instruções***.

2.1 – Introdução

3

- Ao estudar como representar instruções iremos nos deparar com o conceito de ***programa armazenado***.
- Consiste na idéia de representar instruções e dados como números e armazená-los na memória do computador.
- Durante a execução de um programa as instruções serão buscados na memória para informar a operação a ser realizada.
- Se essa operação altera ou usa dados armazenados esses também serão buscados pelo processador.

2.1 – Introdução

4

- Ao projetar um conjunto de instruções algumas metas devem estar em mente:
 - ▣ Encontrar um linguagem que simplifique a implementação do *hardware* e o do compilador
 - ▣ Maximizar performance
 - ▣ Minimizar custo e consumo de energia.

2.2 – Operações do hardware do computador

5

- Nessa seção serão introduzidas algumas instruções da linguagem assembly do MIPS (Microprocessor without Interlocked Pipeline Stages)

- Operações aritméticas

```
add a, b, c
```

- Instrui o computador a adicionar as variáveis *b* e *c* e colocar o resultado em *a*.
- As instruções aritméticas do MIPS realizam apenas uma operação e devem ter exatamente 3 (variáveis) operandos.

2.2 – Operações do hardware do computador

6

- Se quisermos somar as variáveis b , c , d e e e por o resultado em a . Precisamos fazer:

```
add a, b, c    #A soma de b e c é posta em a.  
add a, a, d    #A soma de b, c e d é posta em a.  
add a, a, e    #A soma de e com b, c e d é posta em a.
```

- São necessárias 3 instruções para somar 4 variáveis.
- Observe que para operações como a adição o número natural de operandos é 3:
 - Os números a serem adicionados e um lugar para por o resultado.

2.2 – Operações do hardware do computador

7

- A exigência de que cada instrução tenha exatamente 3 operandos está de acordo com a filosofia de manter o *hardware* simples.
- Já que é mais simples implementá-lo para um número fixo, que para um número variável de operandos.
- Princípio de projeto 1:
 - ▣ Simplicidade favorece a regularidade.

2.2 – Operações do hardware do computador

8

- Exemplo (compilando 2 atribuições em C para MIPS)
- Como traduzir as instruções abaixo para MIPS?

```
a = b + c;  
d = a - e;
```

- Solução: Como as operações do MIPS operam sobre 2 operandos fontes e um destino, duas instruções simples são necessárias:

```
add a, b, c  
sub d, a, e
```


2.2 – Operações do hardware do computador

9

- Exemplo (Compilando uma atribuição complexa, em C, para MIPS)
- Considere a atribuição envolvendo 5 variáveis, abaixo:

```
f = (g + h) - (i + j);
```

- O que o compilador produziria?
 - ▣ O compilador deverá “quebrar” esta sentença em várias instruções *assembly*, uma vez que cada instrução MIPS realiza apenas uma operação.

2.2 – Operações do hardware do computador

10

- ▣ A primeira instrução MIPS calcula a soma de *g* e *h*.

```
add t0, g, h    #t0(temp.) guarda g + h.
```

- ▣ A próxima instrução é análoga e opera sobre *i* e *j*.

```
add t1, i, j    #t1(temp.) guarda i + j.
```

- ▣ Finalmente um instrução de subtração finaliza a atribuição do resultado para *f*.

```
sub f, t0, t1   #f recebe o resultado.
```

2.3 – Operandos do *hardware* do computador

11

- Diferentemente das ling. de alto nível, os operandos das instruções aritméticas são restritos.
- Eles devem ser escolhidos dentre uma lista de estruturas de armazenamento de informação implementadas diretamente em *hardware* chamadas **registradores**.
- O tamanho de um registrador na arquitetura MIPS é 32 bits.
- Grupos de 32 bits ocorrem tão frequentemente que são denominados **palavras**.

2.3 – Operandos do *hardware* do computador

12

- O conjunto de registradores, usados para armazenar os valores das variáveis, é limitado, no MIPS esse número é de 32 registradores de 32 bits.
- A razão para limitar o número de registradores a 32 foi baseada no seguinte princípio:
- Princípio de projeto 2:
 - ▣ Menor é mais rápido.

2.3 – Operandos do *hardware* do computador

13

- Esse número não é absoluto, mas as motivações desse princípio são:
- O número de registradores influencia na distância que os sinais devem percorrer e isso impacta a escolha da frequência de operação.
- Tem que haver um equilíbrio entre:
 - ▣ ter registradores suficientes para os programas
 - ▣ e manter um clock rápido.
- Mais registradores demandam mais bits de identificação (endereço) nas instruções, aumentando seu tamanho.

2.3 – Operandos do *hardware* do computador

14

- Poderíamos usar números entre 0 e 31 para identificar os registradores no código *assembly*.
- No entanto, a notação utilizada pelo MIPS para os registradores segue a seguinte convenção:
 - ▣ Um caracter \$ seguido de um nome de 2 caracteres, por exemplo \$t0.
 - ▣ Usaremos \$s0, \$s1, ... para registradores que correspondem a variáveis no código C
 - ▣ E \$t0, \$t1, ... para registradores temporários usados na compilação.

2.3 – Operandos do *hardware* do computador

15

- Assim a compilação do trecho do exemplo anterior,

```
f = (g + h) - (i + j);
```

- Ficaria,

```
add $t0, $s1, $s2 # $t0 = g + h
```

```
add $t1, $s3, $s4 # $t1 = i + j.
```

```
sub $s0, $t0, $t1 # $s0 = $t0 - $t1.
```

2.3 – Operandos do *hardware* do computador

16

- ❑ **Operandos na memória**
- ❑ As linguagens de programação além de simples variáveis, que armazenam um único dado, podem oferecer estruturas de dados complexas, por exemplo vetores e estruturas.
- ❑ Essas estruturas podem conter um número de elementos muito superior ao número de registradores do computador.
- ❑ Como o computador representa e acessa estas estruturas?

2.3 – Operandos do *hardware* do computador

17

- Vamos relambrar os componentes básicos de um computador:
 - ▣ Entrada – Fornece uma maneira de alimentar o computador com informação.
 - ▣ Saída - Fornece uma maneira de compartilhar os resultados produzidos pelo computador com o usuário ou outro computador.
 - ▣ Memória – Área de armazenamento onde são mantidos os programas quando estão executando e os dados que os programas necessitam.

2.3 – Operandos do *hardware* do computador

18

- ▣ Caminho de dados – Formado pelos componentes do processador que realizam operações aritméticas.
- ▣ Controle – Formado pelos componentes do processador que comandam o caminho de dados, memória e dispositivos de E/S de acordo com as instruções de um programa.
- ▣ OBS: No livro do Stallings, caminho de dados e controle juntos correspondem ao componente processador. Entrada e saída são vistos como um único elemento. O componente de interconexão é tratado explicitamente enquanto aqui fica implícito.

2.3 – Operandos do *hardware* do computador

19

- Podemos perceber então que o processador só pode manter uma pequena quantidade de dados nos registradores.
- Já a memória pode conter um grande número de dados e instruções.
- Logo, estas estruturas, vetores e estruturas, são mantidas na memória.

2.3 – Operandos do *hardware* do computador

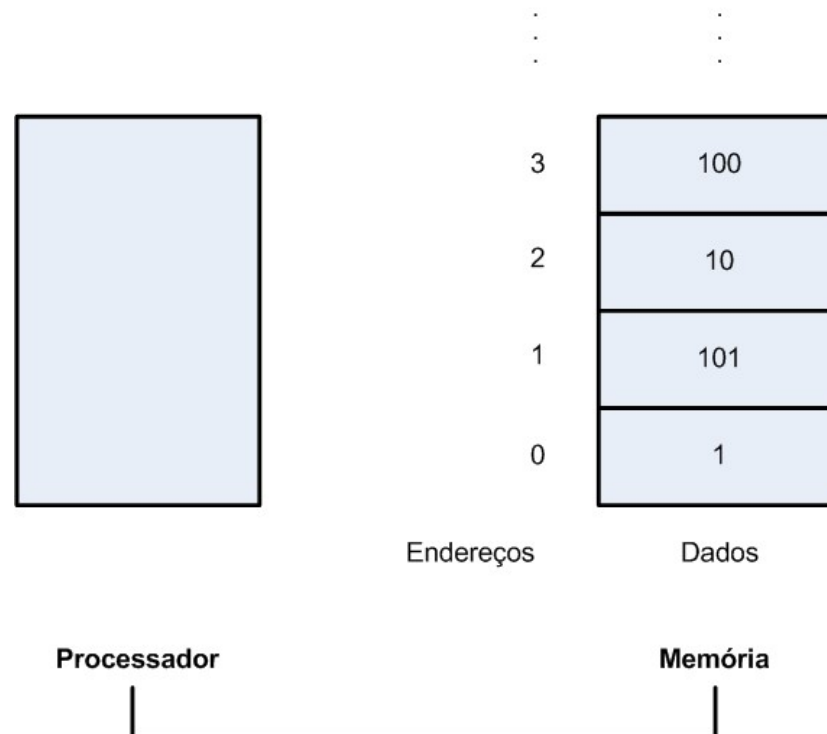
20

- Como explicado anteriormente, as operações aritméticas ocorrem apenas em registradores.
- Desta forma, o MIPS precisa de instruções para transferir dados entre memória e registradores.
- Essas instruções são chamadas ***instruções de transferência de dados***.
- Para acessar uma palavra na memória a instrução deve fornecer um ***endereço*** de memória.

2.3 – Operandos do *hardware* do computador

21

- A memória é como um enorme vetor, com os endereços atuando como os índices para suas posições. Os endereços iniciam em 0.



2.3 – Operandos do *hardware* do computador

22

- A operação de transferência que copia dados da memória para os registradores é chamada **load**, no *assembly* do MIPS `lw` (*load word*).
- O formato da instrução *load* é o seguinte:
 - ▣ Nome da instrução (`lw`), seguido pelo registrador onde o dado será carregado (`$t1`), seguido de uma constante (5) e do registrador usado para acessar a memória (`$s2`).
 - ▣ Ex: `lw $t1, 5 ($s2)` .
- A constante é chamada **offset** ela é adicionada ao valor do **registrador base** (`$s2`) para formar o endereço do dado.

2.3 – Operandos do *hardware* do computador

23

- Exemplo (Compilando uma atribuição quando o operando está na memória)
 - ▣ Suponha um vetor A que armazena 100 palavras.
 - ▣ Assuma que o compilador associou as variáveis g e h com os registradores \$s1 e \$s2.
 - ▣ Considere também que o endereço inicial ou endereço base do vetor está no registrador \$s3.
 - ▣ Compile a seguinte sentença em C:

```
g = h + A[8];
```

2.3 – Operandos do *hardware* do computador

24

□ Solução:

- A primeira coisa a observar é que teremos que trazer o dado armazenado em $A[8]$ para um registrador.
 - O endereço desse elemento do vetor é dado pela soma do valor do registrador base $\$s3$ com o *offset* 8.
 - Esse dado deve ser colocado num registrador temporário para posterior uso pela instrução de adição.
 - Assim,
- OBS: veremos adiante que o offset 8 não produziria o resultado desejado, devido ao alinhamento de memória.

```
lw $t0, 8($3) # $t0 recebe A[8]
```


2.3 – Operandos do *hardware* do computador

25

- ▣ A próxima instrução agora pode usar o valor de $A[8]$, que já está num registrador, portanto, basta adicionar $\$s2(h)$ a $\$t0(A[8])$ e guardar em $\$s1(g)$.
- ▣ Logo,

```
add $s1,$s2,$t0 #g = h + A[8]
```

- ▣ Foi necessário uma instrução de transferência e uma instrução aritmética para realizar a atribuição desejada.

2.3 – Operandos do *hardware* do computador

26

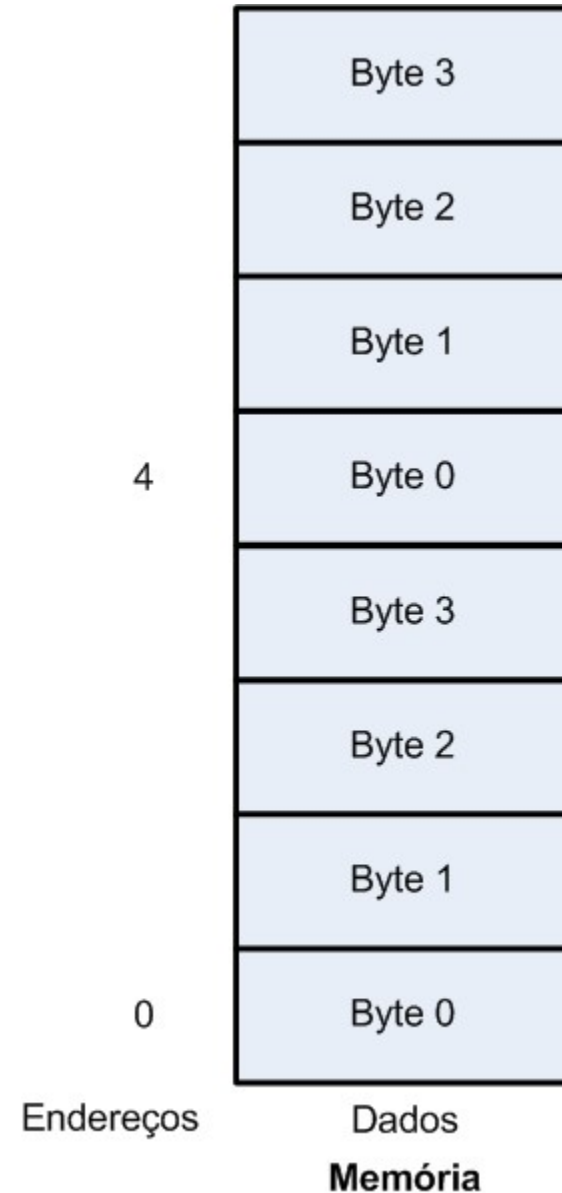
- Vimos que do mesmo modo que variáveis são associadas com registradores, o compilador aloca estruturas de dados, tais como vetores, em localidades de memória.
- Com isso, o compilador sabe que endereço base deverá ser usado nas instruções de transferência de dados.

2.3 – Operandos do *hardware* do computador

27

- A maioria das arquiteturas permitem o endereçamento de bytes (8bits) individualmente.
- Desta forma, o endereço de uma palavra corresponde ao endereço de um dos 4 bytes que formam a palavra.
- Portanto, endereços consecutivos de **palavras** diferem entre si pelo valor 4.

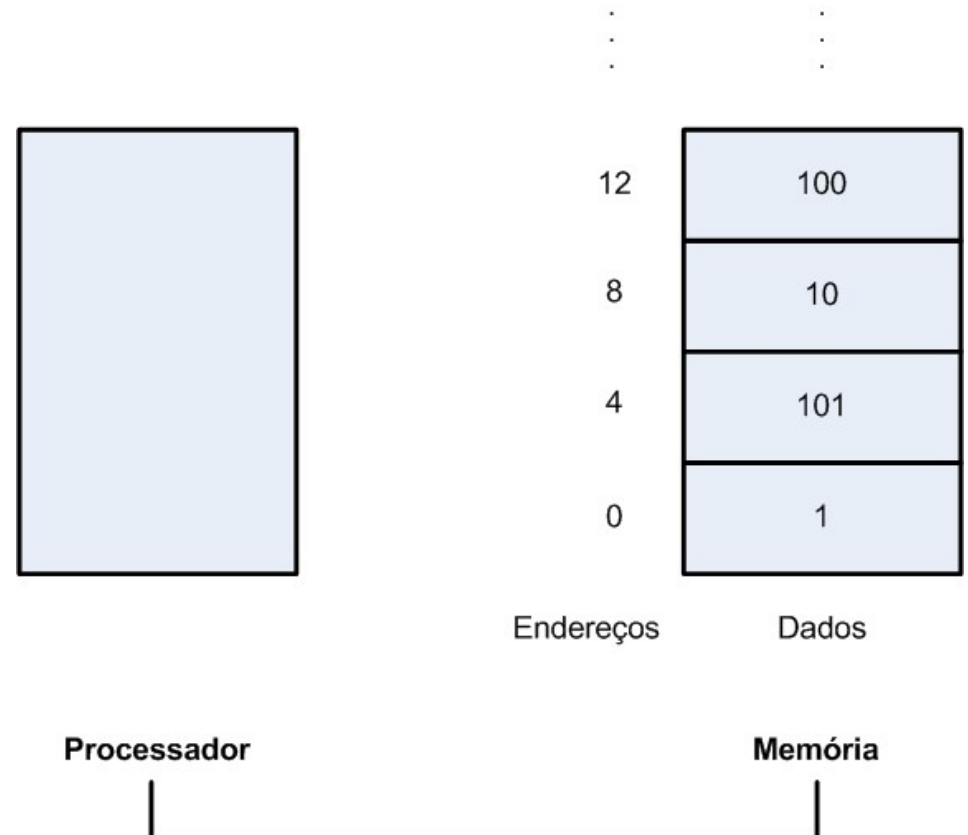
- Se a palavra pode ser vista como um número com esta ordem de bytes: 3, 2, 1 e 0 (menos significativo).
- Então a memória ao lado tem ordenação **little-endian**.
- Se a ordem, no armazenamento, se invertesse a organização seria **big-endian**.



2.3 – Operandos do *hardware* do computador

29

- No MIPS as palavras começam em endereços múltiplos de 4.
- Isso é chamado **restrição de alinhamento**.
- Considerando este detalhe o *offset* do exemplo anterior seria 32 (4 X 8) e não 8.
- Assim, acessaríamos $A[8]$ e não $A[8/4]$.



2.3 – Operandos do *hardware* do computador

30

- Uma instrução complementar ao *load* é o ***store***, que copia dados dos registradores para a memória. No *assembly* do MIPS `SW` (*store word*).
- O formato da instrução *store* é o seguinte:
 - ▣ Nome da instrução(`SW`), seguido pelo registrador onde está o dado a ser armazenado na memória(`$t3`), seguido do *offset* (`9`) e do registrador base(`$s0`).
 - ▣ Ex: `SW $t3, 9($s0)` .

2.3 – Operandos do *hardware* do computador

31

- Exemplo (compilando usando *load* e *store*)
 - ▣ Assuma que `h` está associado como registrador `$s2` e que o registrador base é `$s3`.
 - ▣ Compile a sentença abaixo, em C, para o *assembly* do MIPS.

```
A[12] = h + A[8];
```

2.3 – Operandos do *hardware* do computador

32

□ Solução:

```
lw $t0, 32($s3) # $t0 = A[8]
```

```
add $t0, $s2, $t0 # $t0 = h + A[8]
```

```
sw $t0, 48($s3) # A[12] = h + A[8]
```

- Observe que 48 (12×4) foi usado como offset para *store*.
- OBS: O MIPS segue a filosofia RISC e apenas *load* e *store* acessam memória. Em outras arquiteturas, x86 por exemplo, outras funções além dessas duas realizam transferência de dados.

2.3 – Operandos do *hardware* do computador

33

- ❑ **Operandos imediatos ou constantes**
- ❑ O uso de constantes em programas é grande, por exemplo incrementar um índice para acessar a próxima posição de um vetor.
- ❑ Se usarmos apenas as instruções que conhecemos até agora para usar uma constante teríamos que acessar a memória:

```
lw $t0, offset_4($s1) # $t0 = constante 4
```

```
add $3, $s3, $t0 # $s3 = $3 + $t0 ($t0 = 4)
```

Obs: end. da constante 4 = $\$s1 + \text{offset_4}$

2.3 – Operandos do *hardware* do computador

34

- Uma alternativa que elimina a instrução `lw` é oferecer versões das operações aritméticas nas quais um dos operandos é uma constante.

```
addi $s3,$s3,4    # $s3 = $s3 + 4
```

- Instruções imediatas ilustram o seguinte princípio de projeto:
 - ▣ Princípio de projeto 3: torne o caso mais comum mais rápido.

2.3 – Operandos do *hardware* do computador

35

- Como operandos constantes ocorrem frequentemente, ao incluir constantes nas operações aritméticas, tornamos as operações mais rápidas e econômicas, pois evitamos ter que carregar as constantes da memória.

2.5 – Representando instruções no computador

36

- As instruções podem ser vistas como:
 - ▣ um número binário formado pela concatenação de vários outros números binários que possuem significado próprio (campos).
- Como os registradores são usados na maioria das instruções os identificadores de registradores devem ser mapeados para números.
 - ▣ Temos `$s0` até `$s7` mapeados nos registradores 16 a 23 e os `$t0` até `$t7` mapeados nos registradores 8 a 15.

2.5 – Representando instruções no computador

37

- Exemplo (traduzindo uma instrução assembly do MIPS em instrução de máquina)
- Considere a instrução:

```
add $t0, $s1, $s2
```

- A representação decimal é:

0	17	18	8	0	32
---	----	----	---	---	----

- E portanto a instrução tem 6 campos.

2.5 – Representando instruções no computador

38

```
add $t0, $s1, $s2
```

0	17	18	8	0	32
---	----	----	---	---	----

- O 1° (0) e o 6° (32) campos, juntos, dizem ao computador que a operação é adição.
- O 2° identifica o primeiro operando fonte (17=\$s1) e o 3° o identifica o segundo operando fonte (18=\$s2).
- O 4° identifica o destino da soma (8=\$t0) e o 5° campo não foi usado nessa instrução e foi colocado em zero.

2.5 – Representando instruções no computador

39

- A instrução pode ser representada com seus campos em binário o que nos dá:

000000	10001	10010	01000	00000	100000
6 bits	5bits	5bits	5bits	5bits	6bits

- A forma como a instrução é montada (disposição dos campos, números de bits, etc.) é chamada ***formato de instrução***.
- Somando o tamanho de cada campo, vemos que o MIPS usa instruções de 32 bits.

2.5 – Representando instruções no computador

40

- Para destinguir da representação *assembly*, chamamos a representação numérica de ***linguagem de máquina*** e a sequência dessas instruções de ***código de máquina***.

2.5 – Representando instruções no computador

41

- Os campos de uma instrução MIPS (*R-format*) são:



- Onde:

- op – Operação básica da instrução **opcode**.
- rs – O primeiro registrador fonte.
- rt – O segundo registrador fonte.
- rd – O registrador de destino.
- shamt – Deslocamento (*Shift Amount*).
- funct – Função. Seleciona a variante específica da operação determinada pelo *opcode*.

2.5 – Representando instruções no computador

42

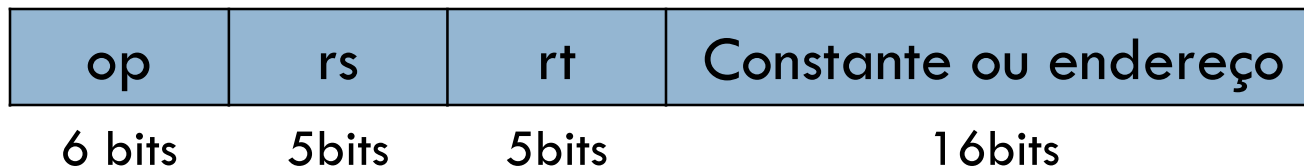
□ Problema:

- ▣ instruções que necessitam de campos com tamanho maior que os descritos anteriormente (*R-Type*).
 - Ex: especificação de constantes na inst. l_w .
 - Se usarmos apenas 5 bits as constantes estarão limitadas ao valor 32 (2^5). Insuficiente para a aplicação.
- ▣ Duas possibilidades de solução:
 - Aumentar o tamanho da instrução
 - Usar formatos de instrução distintos (MIPS)
 - R-format (Register), I-format (Immediate), J-format (Jump), FR-format (Float-point Register), FI-format (Float-point Immediate)

2.5 – Representando instruções no computador

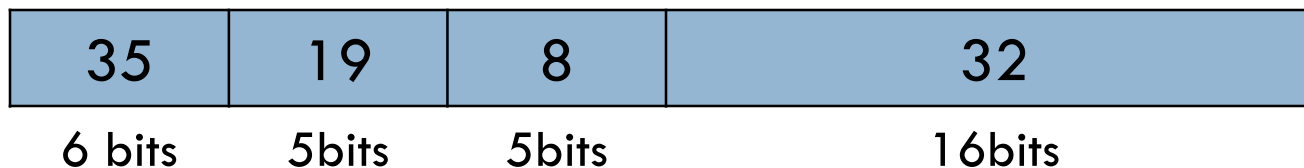
43

□ I-format (32 bits no total)



□ Ex: `lw $t0, 32($s3)`

- Como vimos `$s3=19`, `$t0=8`, constante=32, opcode=35. Assim:



Obs: Nas instruções I-format rt assume a função de rd e indica o destino do resultado.

2.5 – Representando instruções no computador

44

- Exemplo (Traduzindo o *assembly* do MIPS para linguagem de máquina.)
- Considerando que $\$t1$ contém a base do *array* *A* e $\$s2$ corresponde a *h*, traduzir a sentença abaixo:

```
A[300] = h + A[300];
```

2.5 – Representando instruções no computador

45

- Solução (compilar para assembly):

```
lw    $t0, 1200($t1) # $t0 = A[300]
add   $t0, $s2, $t0   # $t0 = h + A[300]
sw    $t0, 1200($t1) # A[300] = $t0
```

- Identificar os formatos, os códigos de operação e de função.

Instruction	Format	op	rs	rt	rd	shamt	funct	address
add	R	0	reg	reg	reg	0	32 _{ten}	n.a.
sub (subtract)	R	0	reg	reg	reg	0	34 _{ten}	n.a.
add immediate	I	8 _{ten}	reg	reg	n.a.	n.a.	n.a.	constant
lw (load word)	I	35 _{ten}	reg	reg	n.a.	n.a.	n.a.	address
sw (store word)	I	43 _{ten}	reg	reg	n.a.	n.a.	n.a.	address

2.5 – Representando instruções no computador

46

□ Mapa dos registradores do MIPS

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

2.5 – Representando instruções no computador

47

- Assim, representando os campos em decimal:

op	rs	rt	rd	address/ shamt	funct
35	9	8	1200		
0	18	8	8	0	32
43	9	8	1200		

- Representando os campos em binário:

100011	01001	01000	0000 0100 1011 0000		
000000	10010	01000	01000	00000	100000
101011	01001	01000	0000 0100 1011 0000		

2.5 – Representando instruções no computador

48

□ Arquitetura MIPS estudada até agora:

Name	Format	Example						Comments
add	R	0	18	19	17	0	32	add \$s1,\$s2,\$s3
sub	R	0	18	19	17	0	34	sub \$s1,\$s2,\$s3
addi	I	8	18	17	100			addi \$s1,\$s2,100
lw	I	35	18	17	100			lw \$s1,100(\$s2)
sw	I	43	18	17	100			sw \$s1,100(\$s2)
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	R	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	I	op	rs	rt	address			Data transfer format

2.6 – Operações lógicas

49

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR			or, ori
Bit-by-bit NOT	~	~	nor

- Se \$s0 contém:

0000 0000 0000 0000 0000 0000 0000 1001_{two} = 9_{ten}

- Se executarmos a instrução `sll $t2, $s0, 4` teremos em \$t2:

0000 0000 0000 0000 0000 0000 1001 0000_{two} = 144_{ten}

2.6 – Operações lógicas

50

- A instrução `sll $t2, $s0, 4` em linguagem de máquina.

0	0	16	10	4	0
op	rs	rt	rd	shamt	funct

- O campo *shamt* agora contém o número de bits que devem ser deslocados.

2.6 – Operações lógicas

51

□ AND

- ▣ Se \$t2 contém:

```
0000 0000 0000 0000 0000 1101 1100 0000two
```

- ▣ E \$t1 contém:

```
0000 0000 0000 0000 0011 1100 0000 0000two
```

- ▣ Se executarmos `and $t0, $t1, $t2` o registrador \$t0 deve conter:

```
0000 0000 0000 0000 0000 1100 0000 0000two
```

2.6 – Operações lógicas

52

- Observe que o AND força os bits de $\$t1$ nas posições onde $\$t2$ contem zeros para zero no resultado.
- Esse uso da operação AND é conhecido como máscara, pois esconde certos bits de $\$t1$ de acordo com o padrão de zeros em $\$t2$ (máscara).

2.6 – Operações lógicas

53

□ OR

▣ Se \$t2 contém:

```
0000 0000 0000 0000 0000 1101 1100 0000two
```

▣ E \$t1 contém:

```
0000 0000 0000 0000 0011 1100 0000 0000two
```

▣ Se executarmos `or $t0, $t1, $t2` o registrador \$t0 deve conter:

```
0000 0000 0000 0000 0011 1101 1100 0000two
```

2.6 – Operações lógicas

54

- NOT (Usando NOR)
- Para manter a idéia de 3 operandos o MIPS não implementa a função NOT.
- O conportamento do NOT é obtido por meio da função NOR.
- Se $\$t1 = 0000\ 0000\ 0000\ 0000\ 0011\ 1100\ 0000\ 0000_{two}$
- Ex: `nor $t0, $t1, $zero` produz em $\$t0$.
 $1111\ 1111\ 1111\ 1111\ 1100\ 0011\ 1111\ 1111_{two}$
- O MIPS oferece as inst. `andi` e `ori` que aceitam operandos imediatos.

2.7 – Instruções para tomar decisões

55

- O que torna um computador diferente de uma calculadora é a capacidade de tomar decisões.
- Baseado em dados de entrada e nos valores produzidos durante a computação, instruções diferentes são executadas.
- Normalmente representado pelo `if` combinado com instruções `goto` em ling. de alto nível.

2.7 – Instruções para tomar decisões

56

- O MIPS oferece 2 instruções de **salto condicional**:

- Saltar se é igual (*branch if equal*):

```
beq register 1, register 2, L1
```

- ▣ Salta para o local indicado pela etiqueta (*label*) L1, se register 1 = register 2.

- Saltar se não é igual (*branch if not equal*):

```
bne register 1, register 2, L1
```

- ▣ Salta para o local indicado pela etiqueta (*label*) L1, se register 1 ≠ register 2.

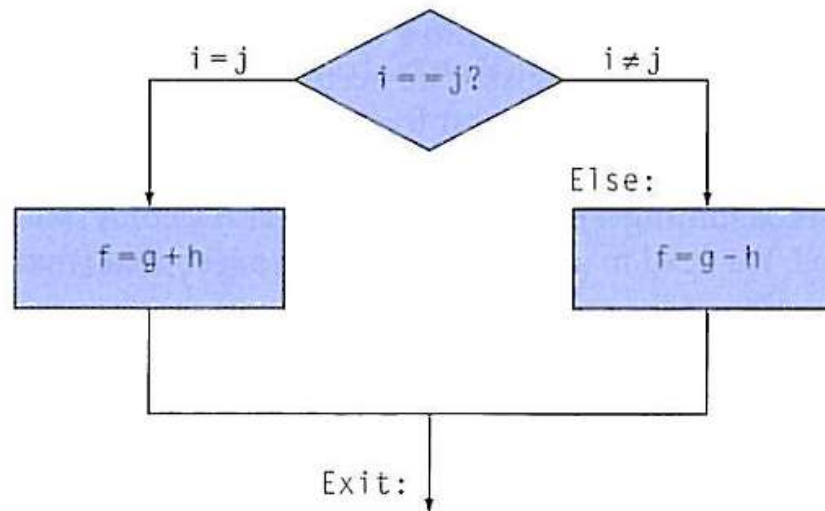
2.7 – Instruções para tomar decisões

57

- Exemplo (Compilando *if-then-else* para saltos condicionais)

```
if (i == j) f = g + h; else f = g - h;
```

- Solução:



2.7 – Instruções para tomar decisões

58

□ Solução:

- Normalmente testar para o caso do *else* torna o código mais eficiente;
- Criar os *labels* para realizar os saltos de acordo com os testes.
- Assim:

```
    bne $s3,$s4,Else    #se i≠j => Else
    add $s0,$s1,$s2     #f=g+h (se i≠j pulado)
    j Exit              #salto incondic. para Exit
Else:
    sub $s0,$s1,$s2     #se i≠j, f=g+h
Exit:
```

2.7 – Instruções para tomar decisões

59

□ Ou seja, segue as formas:

```
        testa condição do senão  
        código do se  
    j Saída  
Senão:  
        código do senão  
Saída:
```

```
        testa condição do se  
        código do senão  
    j Saída  
Se:  
        código do se  
Saída:
```

2.7 – Instruções para tomar decisões

60

□ Laços (*Loops*)

- ▣ As instruções de tomada de decisão são importantes também para o procedimento de iteração.
- ▣ São portanto, os blocos básicos para execução de laços.

□ Exemplo:

```
while (save[i] == k)
    i += 1;
```

- Se *i*, *k* e a base para *save* estão relacionados com \$s3, \$s5 e \$s6 respectivamente.
- Como seria trecho acima em *assembly*?

2.7 – Instruções para tomar decisões

61

□ Solução:

- Temos que calcular o *offset* e o endereço de `save[i]`.
- Carregar `save[i]` para um registrador.
- Testar se `save[i] ≠ k` e de acordo com o resultado realizar `i+=1` ou encerrar.

Loop:

```
sll $t1,$s3,2      # $t1=i*4 (offset)
add $t1,$t1,$s6     # $t1=$s6+$t1 (endereço)
lw  $t0,0($t1)      # $t0=save[i]
bne $t0,$s5,Exit    # Se save[i]≠k => Exit
addi $s3,$s3,1      # $s3=$s3+1 (incremento)
j   Loop            # Salta de volta para Loop
```

Exit:

```
# Termina
```

2.7 – Instruções para tomar decisões

62

□ Exercício: Compilar para MIPS

```
for (i=0, i<10, i++) {  
    a[i] += 5;  
}
```

□ Considere

- `i=$s1;`
- `$s6=base de a[].`
- Use os temporários que necessitar.

2.7 – Instruções para tomar decisões

63

□ Solução:

```
li $t0, 10          #t0=10 (addi $t0,$t0,10)
li $s1, 0           # i=0  (addi $s1,$s1,0)
loop:
    beq $s1, $t0, end # se i==10 termina
    {
        sll $t2, $s1, 2 # $t2=i*4 (offset)
        add $t2, $t2, $s6 # $t2=$s6+$t2 (endereço)
        lw $t3, 0($t2)   # $t3=a[i]
        addi $t3, $t3, 5 # #t3+=5 (inc. de a[i])
        sw $t3, 0($t2)   #
        addi $s1, $s1, 1 # i+=1 (inc. de i)
        j loop           # Salta pra o teste
    }
end:
```

2.7 – Instruções para tomar decisões

64

- Testando a desigualdade do tipo menor que (*less than*)
- No MIPS essa comparação é realizada por uma instrução que compara dois registradores e guarda em um terceiro registrador:
 - 1 se o primeiro é menor que o segundo;
 - e 0 (Zero) para o caso oposto.

```
slt $t0, $s3, $s4 # $t0 = 1 se $s3 < $s4
```

- `slt` (*set on less than*).

2.7 – Instruções para tomar decisões

65

- Existe também a versão com operando imediato

```
slti $t0, $s2, 10 # $t0 = 1 se $s2 < 10
```

- Como existem números com sinal as instruções de comparação têm que lidar com isso, para tanto, temos as instruções:
 - ▣ `slt` e `slti`, que operam com números com sinal
 - ▣ `sltu` e `sltui`, que operam com números sem sinal

2.7 – Instruções para tomar decisões

66

□ Exemplo:

□ Se `$s0` contêm:

1111 1111 1111 1111 1111 1111 1111 1111_{two}

□ E `$s1` contêm:

0000 0000 0000 0000 0000 0000 0000 0001_{two}

□ Qual são os valores de `$t0` e `$t1` após as seguintes instruções:

```
slt  $t0, $s0, $s1 # comparação com sinal
sltu $t1, $s0, $s1 # comparação sem sinal
```

2.7 – Instruções para tomar decisões

67

□ Para $\$S0$:

▣ No caso de ser tratado como um número com sinal

■ Valor = -1_{10} (use as regras de compl. de 2 para converter para decimal)

▣ No caso de ser tratado como um número sem sinal

■ Valor = $4.294.967.295_{10}$

□ Para $\$S1$:

▣ Em ambos os casos:

■ Valor = 1_{10}

□ Logo $\$t0 = 1$ e $\$t1 = 0$.

2.7 – Instruções para tomar decisões

68

- Desta forma, tratar números com sinal como se eles não o fossem nos dá uma maneira eficiente de testar se $0 \leq x < y$.
- Essa condição corresponde ao testes de índice fora dos limites para vetores.
- Assim para fazermos o teste:
 - ▣ se `$s1` (índice) e `$t2` (tamanho)

```
sltu $t0, $s1, $t2 # $t0=0 se $s1>=$t2 ou $s1 < 0  
beq $t0, $zero, Erro # se $t0=0 salta para o Erro
```

2.8 – Suporte a procedimentos no *hardware* do computador

69

- Procedimento (ou função)
 - ▣ Subrotina (subprograma) que realiza uma tarefa específica baseada nos parâmetros que lhe foram passados.

2.8 – Suporte a procedimentos no *hardware* do computador

70

- Para poder executar (“chamar”) um procedimento devemos seguir alguns passos:
 1. Colocar os parâmetros em um lugar no qual o procedimento possa acessá-los;
 2. Transferir o controle para o procedimento;
 3. Adquirir recursos de armazenamento necessários para o procedimento;
 4. Realizar a tarefa desejada
 5. Colocar o resultado em um lugar onde o programa que “chamou” o procedimento possa acessá-lo.
 6. Retornar o controle para o ponto de origem, já que um procedimento pode ser chamado de vários pontos em um programa.

2.8 – Suporte a procedimentos no *hardware* do computador

71

- Registradores usados em uma chamada a procedimento:
 - `$a0–$a3` registradores de argumentos, nos quais são passados os parâmetros.
 - `$v0–$v1` registradores de retorno, onde os valores de retorno produzidos pelo procedimento são passados.
 - `$ra` registrador de endereço de retorno, que guarda o ponto de retorno ao programa “chamador”.

2.8 – Suporte a procedimentos no *hardware* do computador

72

- Em adição a esses registradores, temos uma instrução para chamar procedimentos.

```
jal endDoProdedimento
```

- A instrução `jal` (*jump and link*) salta para o endereço indicado e simultaneamente salva o endereço da instrução seguinte a `jal` no registrador `$ra`.
- O endereço em `$ra` é conhecido como **endereço de retorno**.

2.8 – Suporte a procedimentos no *hardware* do computador

73

- O endereço de retorno é necessário pois o procedimento pode ser chamado de vários pontos do programa.
- A instrução usado para retornar ao ponto de origem é `jr` (*jump register*) que recebe como parâmetro `$ra`.
- Assim:
 - ▣ O programa “chamador” coloca os parâmetros em `$a0–$a3` e usa `jal x` para chamar o procedimento.
 - ▣ O procedimento (“chamado”) realiza sua tarefa, coloca os resultados em `$v0–$v1` e usa `jr $ra` para retornar.

2.8 – Suporte a procedimentos no *hardware* do computador

74

- Sabemos que existe um registrador chamado:
 - **contador de programa** que aponta para a instrução que está sendo executada (antes de ser incrementado na execução da instrução corrente, quando passa a apontar para a próxima instrução)
- Logo, o endereço de retorno salvo em `$ra` pela instrução `jal` é na verdade $PC + 4$.

2.8 – Suporte a procedimentos no *hardware* do computador

75

- **Usando mais registradores**
- Se forem necessários mais registradores que os 4 de argumentos e 2 de retorno, precisaremos usar os registradores `$s0–$s7` e `$t0–$t9`.
- Dessa forma, precisaremos guardar os valores anteriores a chamada do procedimento na memória e restaurá-los ao final do procedimento de forma que o chamador, não perca informação.

2.8 – Suporte a procedimentos no *hardware* do computador

76

- A estrutura de dados ideal para isso é a pilha:
 - ▣ Pilha – uma fila do tipo o último a entrar é o primeiro a sair ou LIFO(*Last In First Out*).
 - ▣ A pilha precisa de um ponteiro para o endereço alocado mais recentemente na pilha que indica onde armazenar e de onde recuperar os valores.
 - ▣ Ele é conhecido como **apontador de pilha** `$sp` (*stack point*)

2.8 – Suporte a procedimentos no *hardware* do computador

77

- Por razões históricas a pilha cresce de endereços altos de memória para endereços baixos, ou seja:
- Coloca-se (*push*) valores na pilha por meio do decremento de $\$sp$.
- E retira-se (*pop*) valores da pilha por meio do incremento de $\$sp$.

2.8 – Suporte a procedimentos no *hardware* do computador

78

□ Exemplo

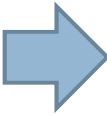
- Um procedimento que não chama outro procedimento (*Leaf Procedures*)
- Considere o código abaixo , em C:

```
int leaf_example (int g, int h, int i, int j)
{
    int f;

    f = (g + h) - (i + j);
    return f;
}
```

2.8 – Suporte a procedimentos no *hardware* do computador

79

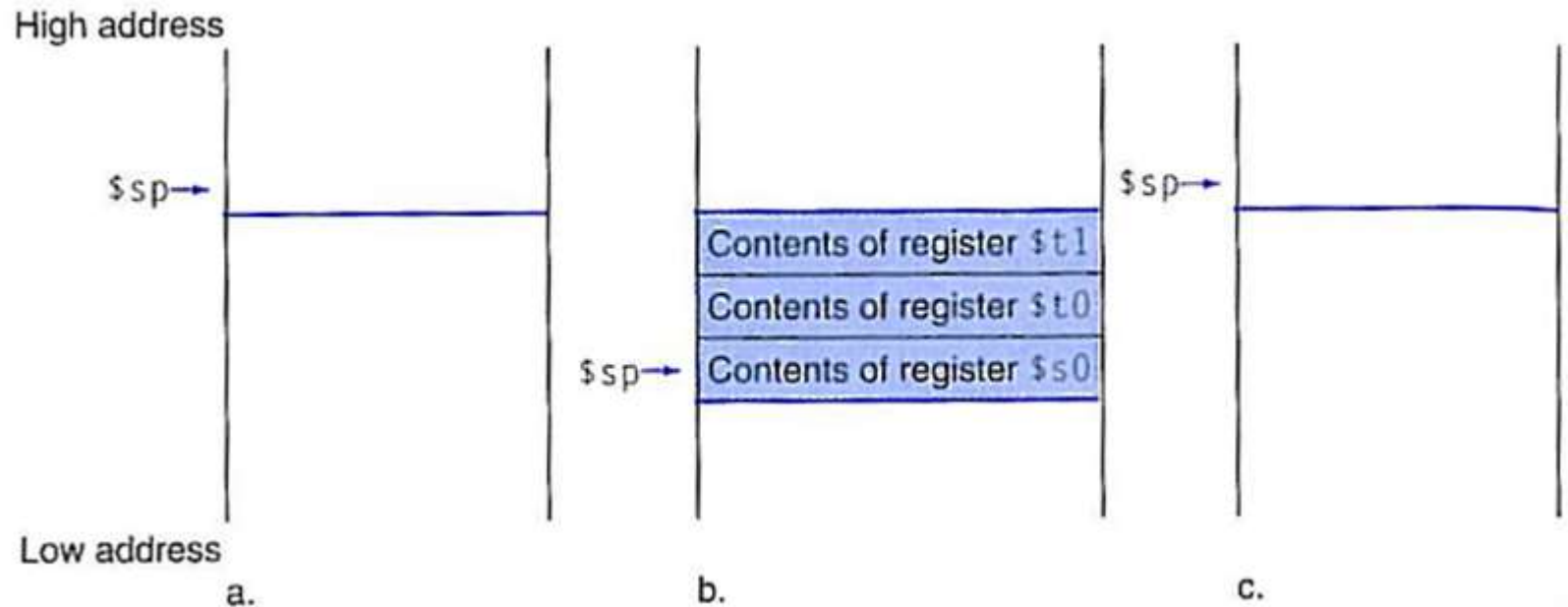
- Qual o código em assembly do MIPS?
 - ▣ Considere $\mathbb{f}$ associado a $\$s0$.
 - ▣ Solução 
 - ▣ Solução modificada 

Obs: Executar o exemplo 3 no QtSpin.

2.8 – Suporte a procedimentos no *hardware* do computador

80

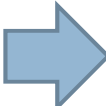
□ Pilha



a) antes, b) durante e c) depois da chamada ao procedimento

2.8 – Suporte a procedimentos no *hardware* do computador

81

- Procedimentos Aninhados (*Nonleaf Procedures*)
 - ▣ Procedimentos que invocam outros procedimentos, inclusive “clones” de si mesmos.
 - ▣ Temos que ter cuidado com o uso de registradores em procedimentos aninhados.
 - ▣ Perigo de perda de end. de retorno e de dados do chamador.
 - ▣ Figura 

2.8 – Suporte a procedimentos no *hardware* do computador

82

□ Exemplo:

- Suponha que um programa principal chama um procedimento A com $\$a0=3$ usando `jal A`.
- Suponha, então, que A chame um procedimento B com $\$a0=7$ usando `jal B`.
- Desde que A ainda não terminou sua tarefa existe um conflito no uso de $\$a0=7$.
- Assim como no endereço de retorno guardado em `$ra`, que agora tem o ponto de retorno de B. Se não tomarmos alguns cuidados esse conflito impedirá que A retorne para o programa principal.

2.8 – Suporte a procedimentos no *hardware* do computador

83

- A solução:
 - ▣ Colocar todos os registradores que precisam ser preservados na pilha. Da mesma forma que fazemos com os registradores $\$s0-\$s7$ (*saved registers*).
 - ▣ Logo, o “chamador” coloca na pilha todos os registradores de argumentos ($\$a0-\$a3$) e os temporários ($\$t0-\$t9$) necessários após o chamada.
 - ▣ O “chamado” coloca o $\$ra$ e todos os registradores $\$s0-\$s7$ (*saved registers*) usados por ele.
 - ▣ O apontador de pilha $\$sp$ é ajustado para refletir a quantidade de resgistradores colocados na pilha.
 - ▣ Após o retorno os valores dos registradores são recuperados da memória e o $\$sp$ reajustado.

2.8 – Suporte a procedimentos no *hardware* do computador

84

□ Exemplo

- ▣ Compilando um procedimento recursivo em C e mostrando a lincagem de procedimentos aninhados.

```
int fact (int n)
{
    if (n < 1) return (1);
    else return (n * fact(n - 1));
}
```

2.8 – Suporte a procedimentos no *hardware* do computador

85

- O programa inicia com o label para o procedimento.
- E salva 2 registradores na pilha o endereço de retorno `$ra` e `$a0`.

```
fact:
    addi $sp, $sp, -8    # Ajusta a pilha para armazenar 2 itens
    sw   $ra, 4($sp)    # Salva o endereço de retorno
    sw   $a0, 0($sp)    # Salva o argumento n
```

2.8 – Suporte a procedimentos no *hardware* do computador

86

- A primeira vez que `fact` é chamado, `sw $ra` salva um endereço da área de memória do programa que o chamou.
- Daí, testa-se se $n < 1$ indo pra L1 se $n \geq 1$.

```
slti $t0, $a0, 1      # Testa se n($a0) < 1
beq  $t0, $zero, L1   # se n>=1, salta para L1
```

- Se $n < 1$, `fact` retorna 1 fazendo `$v0=1`.
- Deleta os registradores salvos na pilha e retorna ao chamador (condição de parada/caso base).

```
addi $v0, $zero, 1    # retorna 1
addi $sp, $sp, 8       # retira os 2 itens da pilha
jr   $ra               # retorna ao chamador
```

2.8 – Suporte a procedimentos no *hardware* do computador

87

- ▣ Antes de deletar os itens da pilha poderíamos tê-los carregado previamente em `$a0` e `$ra`.
- ▣ Porém, como esses valores não mudam quando $n < 1$, não é necessário adicionar essas instruções.
- ▣ Continuando:
- ▣ Se $n \geq 1$, teremos de decrementar o argumento n e chamar `fact (n - 1)`.

L1:

```
addi $a0, $a0, -1    # n>=1: argumento será n-1
jal  fact             # chama fact(n-1)
```

2.8 – Suporte a procedimentos no *hardware* do computador

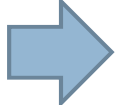
88

- Agora vamos programar o retorno de `fact`.
(retorno geral)
- Nesse caso, temos de restaurar `$a0` e `$ra`. Ajustar o `$sp` (deletar os 2 itens restaurados)
- Calcular o valor de retorno do procedimento
- Retornar ao ponto de origem do chamador

```
lw    $a0, 0($sp)      # restaura argumento n
lw    $ra, 4($sp)      # restaura o endereço de retorno
addi  $sp, $sp, 8      # ajusta $sp, deleta 2 itens
mul   $v0, $a0, $v0    # retorna n * fact( n - 1)
jr    $ra              # retorna ao chamador
```


2.8 – Suporte a procedimentos no *hardware* do computador

89

- Qual o código em assembly do MIPS?
 - ▣ Considere n associado ao reg. de argumentos $\$a0$.
 - ▣ Solução Completa 
- O que é preservado durante uma chamada a procedimento.

Preservado	Não preservado
Saved registers: $\$s0-\$s7$	Temporary registers: $\$t0-\$t9$
Stack pointer register: $\$sp$	Argument registers: $\$a0-\$a3$
Return address register: $\$ra$	Return values registers: $\$v0-\$v1$
Pilha acima do $\$sp$	Pilha abaixo do $\$sp$

2.8 – Suporte a procedimentos no *hardware* do computador

90

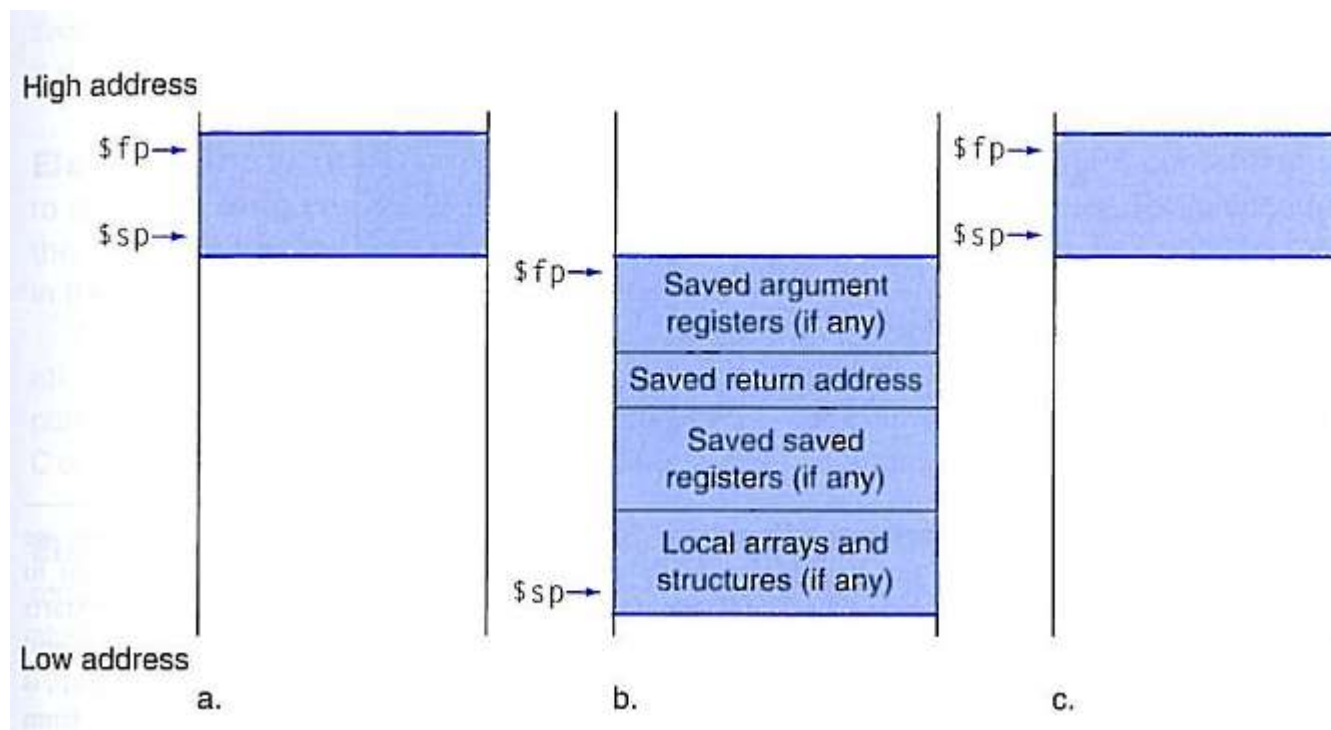
□ **Alocando espaço para novos dados na pilha**

- A complexidade final é que a pilha também é usada para armazenar variáveis locais ao procedimento e que não cabem em um registrador.
- O segmento da pilha que contém os registradores salvos pelo procedimento e suas variáveis locais é conhecido como:
 - **Registro de ativação (*activation record*)**
 - **Ou quadro do procedimento (*procedure frame*)**

2.8 – Suporte a procedimentos no *hardware* do computador

91

□ Uso do *frame point* (\$fp):

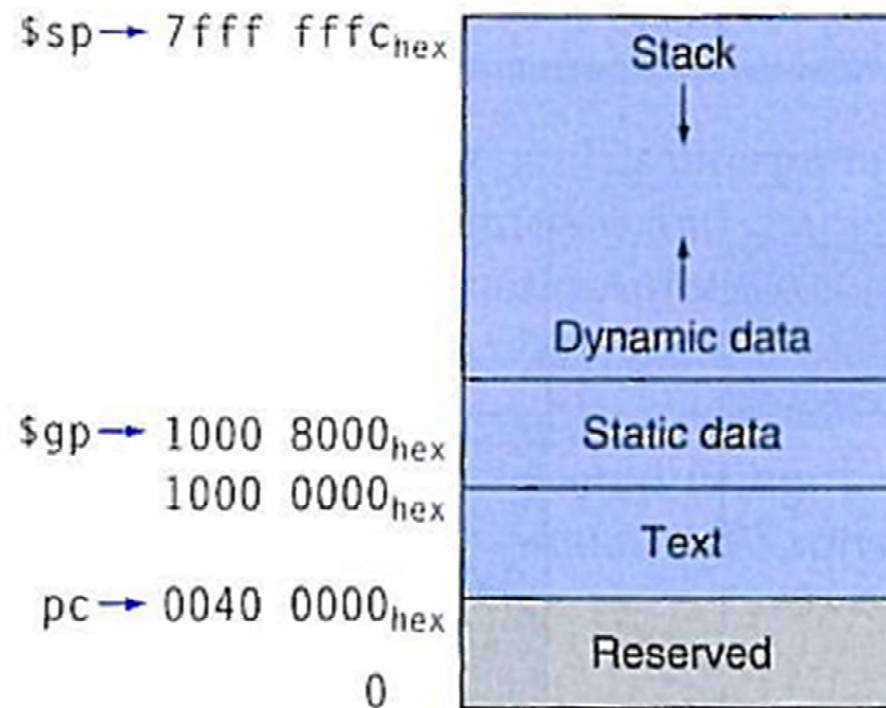


a) antes, b) durante e c) depois da chamada ao procedimento

2.8 – Suporte a procedimentos no *hardware* do computador

92

- Alocação de memória para programa e dados:



2.8 – Suporte a procedimentos no *hardware* do computador

93

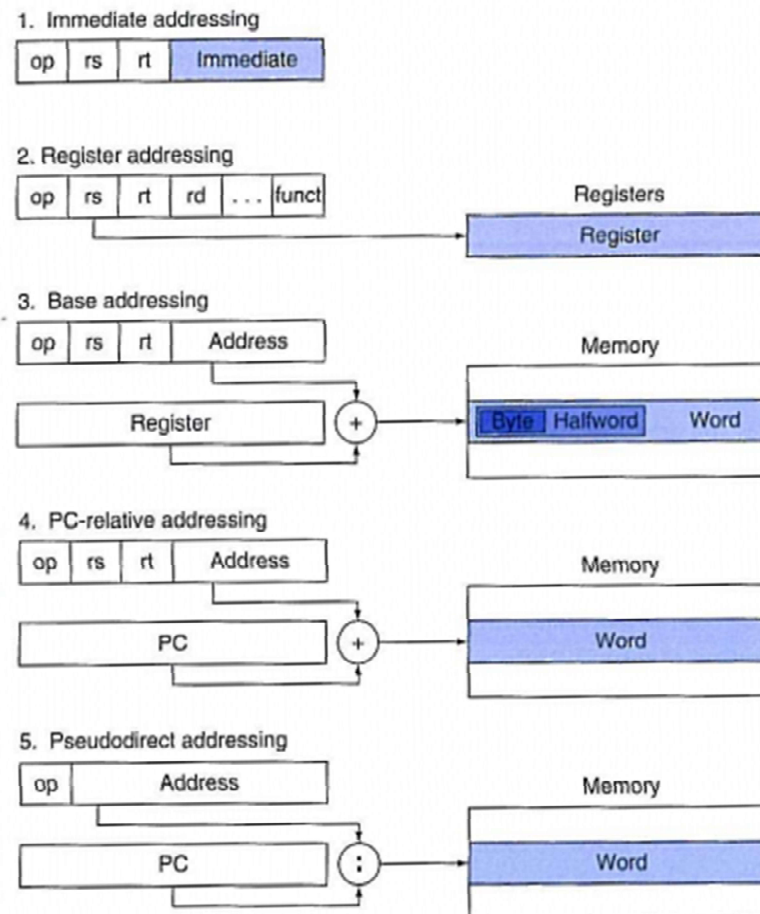
□ Convenção dos registradores MIPS

Nome	Número	Uso	Preservado numa chamada
\$zero	0	Constante 0	n.a.
\$v0-\$v1	2-3	Valores de retorno e avaliação de expressões	Não
\$a0-\$a3	4-7	Argumentos	Não
\$t0-\$t7	8-15	Temporários	Não
\$s0-\$s7	16-23	Saved	Sim
\$t8-\$t9	24-25	Mais temporários	Não
\$gp	28	Ponteiro global (global point)	Sim
\$sp	29	Ponteiro de pilha (stack point)	Sim
\$fp	30	Ponteiro de quadro (frame point)	Sim
\$ra	31	Endereço de retorno	Sim

2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

94

□ Sumário dos modos de endereçamento do MIPS



2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

95

□ São cinco:

▣ Endereçamento imediato:

- O operando é uma constante dentro da própria instrução.

▣ Endereçamento de registrador:

- Quando o operando é um registrador.

▣ Endereçamento por deslocamento ou Base:

- Quando o operando está numa localização de memória cujo endereço é a soma de um registrador (base) com a constante presente na instrução.

2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

96

- Continuando com os modos de endereçamento:
 - ▣ Endereçamento PC-relativo:
 - Quando o endereço do desvio é a soma do PC com uma constante presente na instrução.
 - ▣ Endereçamento pseudodireto:
 - Quando o endereço do salto são os 26 bits presentes na instrução (J-Format) concatenados com os 4 bits mais significativos do PC.

2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

97

□ Decodificando a linguagem de máquina

- ▣ Qual sentença em linguagem assembly corresponde a essa instrução de máquina?

00af8020hex

- O primeiro passo é transformar em binário e descobrir o campo (op);
- com essa informação saberemos o formato (R, I ou J) da instrução e conseqüentemente quais os demais campos e seus tamanhos.

```
(Bits:31 28 26                                     5  2 0)
      0000 0000 1010 1111 1000 0000 0010 0000
```

2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

98

- $op \ (31-26) = 000000_2 \Rightarrow 0_{10} \Rightarrow 0_{16}$, logo a instrução é R-format.
- Assim, proseguimos com a divisão dos demais campos seguindo o formato identificado.

op	rs	rt	rd	shamt	funct
000000	00101	01111	10000	00000	100000

- $funct \ (5-0) = 100000_2 \Rightarrow 32_{10} \Rightarrow 20_{16}$ portando a operação é add (ver *gree sheet*).
- $shamt \ (10-6) = 00000_2$, não há deslocamento de bits.
- $rd \ (15-11) = 10000_2 \Rightarrow 16_{10}$, que corresponde a \$s0.

2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

99

- Continuamos seguindo esse procedimento para os demais campos e concluimos:
 - $rt \ (20-16) = 01111_2 \Rightarrow 15_{10}$, que corresponde a $\$t7$.
 - $rs \ (25-21) = 00101_2 \Rightarrow 5_{10}$, que corresponde a $\$a1$.
 - Assim temos:

```
add $s0, $a1, $t7
```

2.10 - Endereçamento MIPS para operandos imediatos de 32 bits e endereços

100

□ Formatos de instrução estudados:

Nome	Campo					Comentário
Tamanho	6 bits	5 bits	5 bits	5 bits	6 bits	Todas têm 32bits
R - format	op	rs	rt	shamt	funct	Formato de inst. aritmética
I - format	op	rs	rt	end./imediato		Transferência , desvio
J - format	op	endereço alvo				Formato da inst. Jump